

CoTree: A Side-Channel Collision Tool to Push the Limits of Conquerable Space

Changhai Ou^{1b}, Debiao He^{1b}, Kexin Qiao^{1b}, Shihui Zheng^{1b}, Siew-Kei Lam^{2b}, *Senior Member, IEEE*,
and Fan Zhang^{1b}, *Member, IEEE*

Abstract—By introducing collision information into divide-and-conquer distinguishers, the existing collision-optimized side-channel attacks transform the given candidate space into a significantly smaller collision space, thus achieving more efficient key recovery. However, the candidates of the first several subkeys shared by collision chains are still repeatedly detected, which happens very frequently and brings huge computational overhead. To alleviate this, we propose a highly efficient collision-optimized attack named collision tree (CoTree). This collision detection tool exploits tree structure to store the chains created from the same subchain on the same branch, thus significantly reducing the storage requirements. It then benefits from the properties of both tree and collisions and exploits a *top-down* tree building procedure and traverses each node only once when detecting their collisions with a candidate of the subkey currently under consideration. Finally, unlike the traditional *top-down* node removal, CoTree launches a *bottom-up* branch removal procedure to remove the chains unsatisfying the collision conditions from the tree after traversing all the considered candidates of this subkey, thus avoiding the traversal of the branches satisfying the collision condition. These strategies make our CoTree significantly alleviate the *repetitive collision detection*, and our experiments verify that it significantly outperforms the existing works.

Index Terms—Collision attack, collision tree (CoTree), key enumeration, side-channel attack (SCA), template attack (TA).

Manuscript received 15 September 2022; revised 14 February 2023; accepted 1 June 2023. Date of publication 21 June 2023; date of current version 22 November 2023. This work was supported in part by the National Natural Research Foundation of China under Grant 62102290 and Grant 62102025; in part by the National Key Research and Development Program of China under Grant 2022YFB3103800; in part by the Fundamental Research Funds for the Central Universities under Grant 2042022rc0021; in part by the Research Foundation of the State Key Laboratory of Cryptology under Grant MMKFKT202223 and Grant MMKFKT202212; and in part by the New 20 Project of Higher Education of Jinan under Grant 202228017. This article was recommended by Associate Editor N. Mentens. (*Corresponding authors: Kexin Qiao; Fan Zhang.*)

Changhai Ou is with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, Hubei, China, and also with the State Key Laboratory of Cryptology, Beijing 100878, China (e-mail: ouchanghai@whu.edu.cn).

Debiao He is with School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China, and also with the Shandong Provincial Key Laboratory of Computer Networks, Qilu University of Technology (Shandong Academy of Sciences), Jinan 250014, China (e-mail: hedeibiao@163.com).

Kexin Qiao is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100811, China, and also with the State Key Laboratory of Cryptology, Beijing 100878, China (e-mail: qiao.kexin@bit.edu.cn).

Shihui Zheng is with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: shihuizh@bupt.edu.cn).

Siew-Kei Lam is with the School of Computer Science and Engineering, Nanyang Technological University, Singapore (e-mail: assklam@ntu.edu.sg).

Fan Zhang is with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, Zhejiang, China (e-mail: fanzhang@zju.edu.cn).

Digital Object Identifier 10.1109/TCAD.2023.3288512

I. INTRODUCTION

SECRET information will unintentionally leak through side channels, such as execution time [11], power consumption [27], electromagnetic radiation [7], and cache patterns [13] when cryptographic algorithms are executed on devices. These side-channel information can be collected by adversaries and they can launch the side-channel attacks (SCAs) in two models: 1) divide-and-conquer (e.g., correlation power analysis (CPA) [4] and template attack (TA) [5]) and 2) analytical (e.g., collision attacks [15]). For the former, they divide the full key into small blocks (e.g., subkeys in AES-128) and conquer them one by one (independently). For the latter, they consider all subkeys simultaneously and recover the key by solving a system of equations. Therefore, the analytical attacks exploit more leaky information and are more efficient than divide-and-conquer attacks, but are also more complex.

It is worth mentioning that the key recovery in the above two models is limited by the computing power of the adversaries. Benefitting from enough side-channel information, if we rank the candidates of each subkey from the most possible one to the least possible one, the correct one is usually ranked the front. However, due to the limited side-channel information, it may not be ranked so well. In this case, key enumeration strategies [10], [12], [21], which enumerate the full-key candidates from the most possible one to the least possible one, are exploited to optimize the key recovery. However, they are still limited by the computing power of the attacker, and can only be exploited in the scenarios that cryptographic implementations are “practically insecure” (for which the leakage allows for key enumeration).

In recent years, several collision-optimized attacks introduced collision information into divide-and-conquer attacks to launch combined attacks (e.g., [3], [18], and [26]), thus establishing the relationship between subkeys and breaking their independence. Unlike key enumeration schemes, they transform the original candidate space into a much smaller collision (chain) space, thus facilitating the later key recovery and achieving better performance than each of them. These collision-optimized attacks will be introduced in the next section before introducing our contributions.

A. Related Works

Collision-optimized attacks can attack the serially implemented cryptographic algorithms with S-box operations like AES, DES, PRESENT, etc. [8]. They rank the candidates

of each collision value in the analytical collision attack and subkey in the divide-and-conquer attack to be optimized from the most possible one to the least possible one. They then set two *thresholds* for these two attacks and only consider the candidates within them. The first work exploiting collision information to optimize the key recovery of divide-and-conquer attacks was given in [3]. Since then, several collision-optimized attacks have been proposed and their goals can be classified into three categories: 1) *collision exploitation*; 2) *fault tolerance*; and 3) *repetitive collision detection alleviation*.

For *collision exploitation*, the first collision-optimized attack named test-of-chain (abbreviated to TOC for convenience here) was given in [3]. Take an encryption algorithm with a total of L S-boxes in one round for an example, TOC exploits the collisions between each two adjacent subkeys, thus $L - 1$ pairs of collisions are exploited and most collisions are wasted. This also happens in [19], which exploits more than $2 \cdot L$ pairs of collisions without fault tolerance. To improve this, full-collision chain (FCC) was given in [17] to quickly exploit all $([L \cdot (L - 1)]/2)$ collisions. It can provide us with the smallest collision space, thus the remaining key recovery is the most efficient. However, any collision value beyond the threshold will lead to the failure of key recovery and this collision condition is very strict. We have to face with very huge (sometimes even unconquerable) collision threshold in this case.

For *fault tolerance*, the first collision-optimized attack with fault tolerance strategy named fault-tolerant chain (FTC) was given in [26]. It exploits the $L - 1$ collisions between the first subkey and the remaining $L - 1$ subkeys and discards the remaining pairs, just like TOC. The fault tolerance here means, if there is no collision between a subkey and the first subkey, FTC enumerates its candidates beyond the threshold from the most possible one to the least possible one until finds a collision. Ou et al. [18] stated that the collision threshold could be significantly reduced by performing fault tolerance on a small number of collision values (i.e., allowing them to be beyond the threshold of collision attack), and provided a flexible fault-tolerant strategy with high performance.

For *repetitive collision detection alleviation*, which happens if several chains share the same candidates for their first several subkeys and the collisions between them and the coming subkey will be repeatedly detected in this case. For example, FCC exploits the collision information between the current subkey and all subkeys on the chain, and repetitive collision detection happens if some chains are built from the same subchain. It is obvious that no repetitive collision detection happens in TOC and FTC, since only a pair of collision between the subkey currently under consideration and a subkey on the chains is exploited. This repetitive collision detection will bring very large computation overhead, and the lightweight collision detection (LCD) algorithm given in [18] exploits a highly efficient jump collision detection mechanism to optimize this. Specifically, LCD considers the longest common subchain in the current chain set and detects collisions between the candidates of the coming subkey and it only once

(see Section III-E for details). In this case, it efficiently reduces repetitive collision detection.

As mentioned above, our goal in this article is how to exploit collisions to optimize the key recovery (i.e., launch an efficient collision-optimized attack), rather than improve the performance of single side-channel distinguishers like the improved collision attack in [2] and the genetic algorithm-based sieve built from CPA in [6]. There are also several algorithms trying to remove a part of candidates to recover the key easier (e.g., the improved CECA in [28]). Although the above works are also interesting and important, they deviate from the purpose of this article. Therefore, we will not discuss them in the remainder of this article.

B. Our Contributions

This article proposes a collision detection algorithm, i.e., a collision-optimized attack tool, named collision tree (CoTree) to more efficiently alleviate the *repetitive collision detections*. Our main contributions are as follows.

- 1) CoTree exploits tree structure to store the long collision chains established from the same short subchain on the same branch, thus avoiding the independent storage of collision chains and significantly saving the storage space.
- 2) Benefiting properties from both tree and collisions, CoTree exploits a *top-down* tree building procedure to significantly alleviate the repetitive collision detection, since the nodes on the tree need to be traversed only once for each candidate (within threshold) of the subkey currently under consideration. This advantage becomes more significant under much larger thresholds.
- 3) Unlike the traditional top-down node removal, CoTree launches a *bottom-up* branch removal procedure to remove the candidates unsatisfying the collision conditions after considering all candidates (within the threshold) of the coming subkey, which avoids repetitive traversal of the branches satisfying the collision condition and makes the complexity smaller than the number of nodes.

Our experiments verify that compared to the existing collision-optimized attacks, our CoTree significantly alleviates the burden from repetitive collision detection and achieves much better performance.

C. Organization

The remainder of this article is organized as follows. TA and CECA are introduced in Section II. The existing collision-optimized attacks, such as TOC, FTC, FCC, and LCD, are presented in Section III. Our CoTree and its optimizations are detailed in Sections IV and V. Experiments on the DPA contest v4.1 dataset and an AT89S52 microcontroller are presented in Section VI to illustrate the superiority of our CoTree. Finally, we conclude this article in Section VII.

II. PRELIMINARIES

Let $\kappa^* = (\kappa_1^*, \kappa_2^*, \dots, \kappa_L^*)$ denote the key of an encryption algorithm with a total of L subkeys (e.g., $L = 16$ for

AES-128), $\kappa = (\kappa_1, \kappa_2, \dots, \kappa_L)$ denote the key guessed by the attacker, and $x^i = (x_1^i, x_2^i, \dots, x_L^i)$ denote the corresponding i th encrypted plaintext. Suppose that the attacker encrypts N plaintexts $\mathbf{X} = (x^1, x^2, \dots, x^N)$ with $\mathbf{X}_j$ denoting their j th bytes, and collects N power traces $\mathbf{T} = (t^1, t^2, \dots, t^N)$. Here, let t_j^i ($i = 1, 2, \dots, N, j = 1, 2, \dots, L$) denote the samples of the i th trace leaking from the operation of the j th S-box. Collision-optimized attacks can be applied to serially implemented cryptographic algorithms with S-box operations like AES, DES, PRESENT etc. For convenience, the remainder of this article takes AES-128 as an example to introduce the related works and our CoTree. Before this, the classic CPA, TA, and CECA are first given in Sections II-A–II-C.

A. Correlation Power Analysis

CPA [4] is a well-known model-dependent distinguisher, for which the attacker needs a hypothesis model, e.g., Hamming weight model, to exploit side-channel leakage information. Here, the corresponding assumed power consumption of the intermediate values under a guessing subkey κ_j can be denoted as: $\mathbf{I}^{k_j} = \text{Hw}(\text{Sbox}(\mathbf{X}_j \oplus \kappa_j))$. CPA calculates the correlation coefficient between it and the real power consumption $\mathbf{T}$ and returns the subkey candidate corresponding to the maximum correlation coefficient

$$\arg \max_{\kappa_j} \{ \rho(\mathbf{I}^{k_j}, \mathbf{T}) | \kappa_j = 0, 1, \dots, 255 \}. \quad (1)$$

Here, “Sbox(·)” denotes the lookup table operation, “Hw(·)” denotes Hamming weight computation, and “ $\rho(\cdot, \cdot)$ ” denotes the Pearson correlation coefficient computation.

B. Template Attack

The classic TA [5] assumes that the attacker has the same device as the targeted one, and it includes two phases: 1) leakage profiling and 2) exploitation. For a cryptographic device with leakage following nine different normal distributions for nine Hamming weights of S-boxes outputs of AES-128, the attacker calculates the mean

$$\mathbf{m}_j^k = \frac{1}{n} \sum_{i=1}^n t_j^i \quad (2)$$

and covariance matrix

$$\mathbf{C}_j^k = \frac{1}{n} \sum_{i=1}^n (t_j^i - \mathbf{m}_j^k)(t_j^i - \mathbf{m}_j^k)' \quad (3)$$

from n ($n < N$) power traces to profile template $(\mathbf{m}_j^k, \mathbf{C}_j^k)$ ($0 \leq k \leq 8$) for Hamming weight k corresponding to the j th S-box in the profiling stage. If the attacker collects a power trace t^* from the targeted device, the probability that it well matches the template $(\mathbf{m}_j^k, \mathbf{C}_j^k)$ corresponding to Hamming weight k is

$$p(t^* | \mathbf{m}_j^k, \mathbf{C}_j^k) = \frac{e^{-\frac{(\mathbf{m}_j^k - t^*) \cdot (\mathbf{C}_j^k)^{-1} \cdot (\mathbf{m}_j^k - t^*)'}{2}}}{\sqrt{(2 \cdot \pi)^{|\mathbf{m}_j^k|} \det(\mathbf{C}_j^k)}}. \quad (4)$$

Here, $|\cdot|$ is the dimension of a variable. S-box operation usually corresponds to many samples and this number increases with

higher sampling rate exploited. To achieve better performance, TA is usually performed on points-of-interest (POIs) with obvious leakage (see [22] for more details).

C. Correlation-Enhanced Collision Attack

An AES-128 linear collision [3] occurs if two S-boxes in the same AES encryption or different encryptions receive the same byte value as their inputs

$$\text{Sbox}(x_{j_1}^{i_1} \oplus \kappa_{j_1}^*) = \text{Sbox}(x_{j_2}^{i_2} \oplus \kappa_{j_2}^*). \quad (5)$$

Let δ_{j_1, j_2}^* denote the exclusive OR value between the j_1 th and j_2 th subkeys, then (5) can be optimized as

$$\delta_{j_1, j_2}^* = \kappa_{j_1}^* \oplus \kappa_{j_2}^* = x_{j_1}^{i_1} \oplus x_{j_2}^{i_2} \quad (6)$$

and we obtain this collision value. Unlike the previous collision attacks judging collisions by comparing the similarity of samples in [23] and [24], CECA divides the samples of the j_1 th and j_2 th S-boxes into 256 classes, respectively, according to their plaintext byte values, then correlates them under a guessing collision value δ_{j_1, j_2} and obtained

$$\rho \left\{ \left(\mathbf{m}_{j_1}^\beta, \mathbf{m}_{j_2}^{\beta \oplus \delta_{j_1, j_2}} \right) | \beta = 0, 1, 2, \dots, 255 \right\}. \quad (7)$$

Here, $\rho\{\cdot\}$ denotes the correlation coefficient computation. Obviously, CECA does not need an assumed leakage model like CPA in Section II-A in this case. This advantage makes CECA well applied in several complex scenarios (e.g., the flawed rotated S-boxes masking (RSM) implementation with first-order leakage in [15]).

III. EXISTING COLLISION-OPTIMIZED ATTACKS

Let τ_d and τ_κ denote the thresholds for the analytical CECA and the divide-and-conquer attack to be combined within collision-optimized attacks, which means we only consider the best τ_κ candidates of each subkey in the divide-and-conquer attack and the best τ_d candidates for each collision value δ_{j_1, j_2} in CECA. The existing collision-optimized attacks are given in Sections III-A–III-E.

A. Collision Chain

A cryptographic system often has a very huge key candidate space to against exhaustion (e.g., 2^{128} in AES-128), thus the adversary cannot traverse all candidates in attacks. This also happens in key enumeration [12], [21], e.g., the adversary may take about one day to enumerate 2^{40} key candidates on a desktop computer. Taking advantage of collision information, the existing collision-optimized attacks transfer the given candidate space within τ_κ into a much smaller collision space to significantly reduce the costs of key recovery. Specifically, if the adversary detects m pairs of collisions, then he obtains a collision system with m linear equations

$$\begin{cases} \kappa_{j_1} \oplus \kappa_{j_2} = \delta_{j_1, j_2} \\ \kappa_{j_3} \oplus \kappa_{j_4} = \delta_{j_3, j_4} \\ \vdots \\ \kappa_{j_{2m-1}} \oplus \kappa_{j_{2m}} = \delta_{j_{2m-1}, j_{2m}} \end{cases} \quad (8)$$

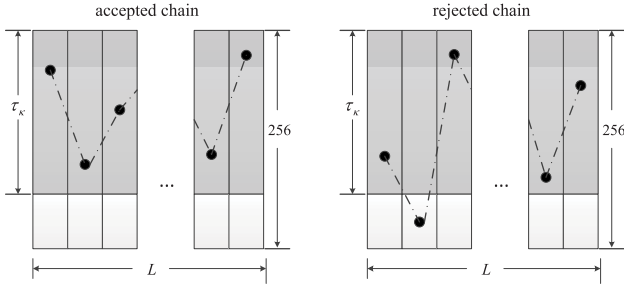


Fig. 1. Guessing chain is rejected if at least one subkey falls outside the most probable τ_k candidates as suggested by the divide-and-conquer attack.

This equation system can be divided into several subsystems with independent variables, and each of them contains collision information of several subkeys and a free variable. They are named as *collision chain*. To recover the key, he only needs to perform brute-force on one subkey for each subsystem, which significantly reduces the candidate space.

B. Test-of-Chain

For convenience, we exploit $\kappa_{j_1} \leftrightarrow \kappa_{j_2}$ to denote the collision between κ_{j_1} and κ_{j_2} , which means these two candidates of the j_1 th and j_2 th subkeys and their guessing collision value δ_{j_1, j_2} are within thresholds simultaneously. TOC detects the collisions $\kappa_1 \leftrightarrow \kappa_2, \kappa_2 \leftrightarrow \kappa_3, \dots, \kappa_{L-2} \leftrightarrow \kappa_{L-1}$ and $\kappa_{L-1} \leftrightarrow \kappa_L$ between any two adjacent subkeys in turn (as shown in Fig. 1). Finally, it obtains chains containing information of all L subkeys. Obviously, TOC exploits only $L - 1$ pairs of collisions and discards the remaining $(L - 1)(L/2 - 1)$ pairs (e.g., exploits 15 pairs in AES-128 and discards the other 105 pairs). It is noteworthy that TOC does not contain any fault-tolerant strategy, a guessing chain is rejected if at least one subkey falls outside its most probable τ_k candidates as suggested by the divide-and-conquer attack. Any collision values falling out of thresholds τ_d will also make it fail.

C. Fault-Tolerant Chain

Different from TOC previously introduced in Section III-B, another collision-optimized attack named FTC in [26] tries to find the collisions $\kappa_1 \leftrightarrow \kappa_2, \kappa_1 \leftrightarrow \kappa_3, \dots, \kappa_1 \leftrightarrow \kappa_{L-1}$ and $\kappa_1 \leftrightarrow \kappa_L$ between the first subkey and the remaining $L - 1$ subkeys. Fault tolerance here means that if there is no collision between κ_1 and the candidates within threshold τ_k of κ_j ($2 \leq j \leq L$), FTC then enumerates κ_j ($2 \leq j \leq L$) beyond τ_k from the most possible to the least possible until finds one. However, this only happens with a very small probability when both τ_k and τ_d are very large, such as the ones considered in this article. Similar to TOC, FTC also exploits only $L - 1$ pairs of collisions and wastes the remaining $(L - 1)(L/2 - 1)$ pairs.

D. Full-Collision Chain

FCC in [17] aims to maximize the use of collision information. It detects the collisions between the current j th ($1 \leq j \leq L$) subkey and all the first $j - 1$ subkeys, and saves the chains with all $j \cdot (j - 1)/2$ pairs of collisions. However, this

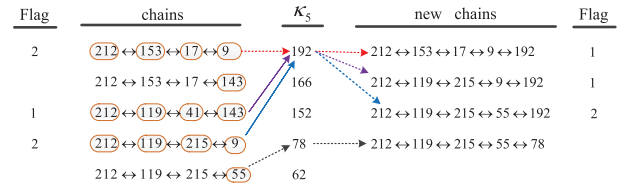


Fig. 2. Jump collision detection mechanism in LCD.

collision condition requiring all subkeys and collision values to be within thresholds τ_k and τ_d is very strict and the authors provided a fault-tolerant strategy to allow a small part of collisions beyond the threshold τ_d . Another shortage of FCC is that it does not contain any mechanism for its *repetitive collision detection*, and too many collisions are repetitively detected. However, TOC and FTC only exploit the collision between the subkey currently under consideration and one of the subkeys on the chains, repetitive collision detection does not exist in this case as we have explained in Section I-A.

E. Lightweight Collision Detection Algorithm

Benefitting from its jump detection mechanism, the LCD algorithm in [18] efficiently alleviates *repetitive collision detection* in FCC. Suppose the current state of collision chain detection is as shown in Fig. 2 and LCD exploits an array “Flag” to record the number of chains established from a collision chain. It can be seen that there are 2, 1, and 2 chains built from subchains $212 \leftrightarrow 153 \leftrightarrow 17$, $212 \leftrightarrow 119 \leftrightarrow 41$, and $212 \leftrightarrow 119 \leftrightarrow 215$, respectively. The collisions built from the same subchains need to be detected only once when considering the five candidates of the newly coming subkey κ_5 within threshold τ_k . LCD jumps over the detection of the corresponding subchains according to the “step length” in array “Flag.” In other words, we only need to detect the candidates on chains with circles on the right. Obviously, the more chains built from a subchain, the longer the step, and the more effective the mechanism is.

IV. OUR CoTREE ALGORITHM

Taking advantage of its jump collision detection strategy, LCD can effectively alleviate the repetitive collision detection. However, limited by its data structure and algorithm design, this problem is still very serious in LCD. In this section, we will give a new algorithm named CoTree to more efficiently alleviate the repetitive collision detection. Trees can use linked structure or array structure to save their data, and each has its advantages and disadvantages. These above two data structures are very convenient for the implementation of our CoTree algorithm, and this section takes array structure as an example to introduce it.

A. Algorithm Description

For array structure, which means CoTree takes father-child presentation and stores the collisions in an array, each node (i.e., a pair of collisions) will take a row. The five fields of each node on this tree are as follows.

- 1) *Candidate*: A candidate of a subkey within threshold τ_k .

Algorithm 1: CoTree Algorithm

Input: Candidates κ -s, collision values δ -s, the number of subkeys L , and thresholds τ_d and τ_κ .
Output: The collision chains $\mathbf{C}$.

```

1  $\mathbf{T}_1 \leftarrow \text{TreeInitialization}(\kappa_1, \tau_\kappa)$ ;
2 for  $j$  from 2 to  $L$  do
3    $\mathbf{T}_j \leftarrow \text{NodeInsertion}(\mathbf{T}_{j-1}, \kappa_j, \delta_j, \tau_d, \tau_\kappa, j)$ ;
4    $\mathbf{T}_j \leftarrow \text{NodeRemoval}(\mathbf{T}_j)$ ;
5   if  $\mathbf{T}_j$  is  $\mathbf{T}_{j-1}$  then
6     break;
7   end
8 end
9  $\mathbf{C} \leftarrow \text{ChainExtraction}(\mathbf{T}_L)$ ;

```

- 2) *Parent*: Identifier (i.e., the row number) of its parent node.
- 3) *Layer*: Layer of this node on the tree. For example, the layer of a candidate of the j th subkey is j .
- 4) *First Child*: Identifier (i.e., the row number) of the first child of this node (if exists).
- 5) *Degree*: The number of branches of this node, i.e., the number of children of this node.

Let $\mathbf{T}_j$ denote a CoTree created from candidates of the first j subkeys within τ_κ , our CoTree exploiting collision between every two subkeys is given in Algorithm 1. It takes candidates κ -s, collision values δ -s, thresholds τ_d and τ_κ as inputs, and outputs the collision chains $\mathbf{C}$.

The main operations of our CoTree include the following.

- 1) *Tree Initialization*: Our CoTree first adds an additional root to make the candidates of the first subkey within τ_κ fall to the second layer as leaves in initialization(step 1). This facilitates the processing of all subkeys when they are considered.
- 2) *Node Insertion*: Considering all collisions between every two subkeys, CoTree inserts candidates of the i th subkey ($2 \leq i \leq L$) as new leaves if they collide with all nodes of a branch from the second layer to leaf (step 3). In other words, they collide with all the candidates of the previous $i - 1$ subkeys on this chain.
- 3) *Node Removal*: CoTree removes the nodes and branches unsatisfying the given collision conditions after traversing all candidates of the current i th subkey under consideration (step 4).
- 4) *Chain Extraction*: CoTree extracts collision chains from the collision tree after finishing the processing of the L th subkey (step 9).

It is noteworthy that if no new node is inserted into the tree after traversing all candidate values of the i th subkey within τ_κ (i.e., $\mathbf{T}_j = \mathbf{T}_{j-1}$), an adversary can consider that key recovery fails and CoTree finishes (steps 5–7). All the above operations (i.e., tree initialization, node insertion, node removal, and chain extraction) in our CoTree will be introduced in detail in Sections IV-B–IV-E.

B. Tree Initialization

Our CoTree is based on the tree structure, and we should first consider how to exploit a collision tree to store collision

Algorithm 2: Node Insertion Operations

Input: Collision tree $\mathbf{T}_{j-1}$, candidates κ_j , collision values δ -s, thresholds τ_d and τ_κ , j .
Output: The updated collision tree $\mathbf{T}_j$.

```

1  $i \leftarrow \text{size}(\mathbf{T}_{j-1})$ ;  $i' \leftarrow i$ ;
2 for  $u$  from 1 to  $\tau_\kappa$  do
3   Initialize an array:  $\mathbf{C}_{1 \dots i} \leftarrow 0$ ;
4   for  $v$  from 2 to  $i$  do
5      $cFlag \leftarrow \text{CheckCollision}(\mathbf{T}_{j-1}, \kappa_j, v, \delta, \tau_d, u)$ ;
6      $\mathbf{C} \leftarrow \text{CountCollision}(\mathbf{C}, \mathbf{T}_{j-1}, v, cFlag)$ ;
7     if  $\text{IsLeaf}(\mathbf{T}_{j-1}, \mathbf{C}, v, j) = 1$  then
8        $\mathbf{T}_{j-1} \leftarrow \text{AddNode}(\mathbf{T}_{j-1}, v, \kappa_j, u, i', j)$ ;
9       if  $\text{NodeDegree}(\mathbf{T}_{j-1}, v) = 1$  then
10         $\mathbf{T}_{j-1} \leftarrow \text{FirstChild}(i', \mathbf{T}_{j-1}, v)$ ;
11      end
12    end
13  end
14 end
15  $\mathbf{T}_j = \mathbf{T}_{j-1}$ ;

```

information (i.e., collision chains). There are two models for tree initialization.

- 1) CoTree detects the possible collisions between the first and second subkeys, and creates a tree for each candidate of the first subkey κ_1 within the threshold τ_κ if it contains at least one collision.
- 2) CoTree adds an additional root for CoTree, and all candidates of κ_1 within τ_κ are directly inserted as the leaves in the second layer, since there is no real collision between the root and the candidates of the first subkey in this case.

There are at most τ_κ and at least 0 trees in our CoTree algorithm in the first model. Actually, all operations of these trees are the same. Therefore, the second model better facilitates our future processing, and we exploit it in our CoTree.

C. Node Insertion

Node insertion in our CoTree is a *top-down* procedure, which ensures that each node on the tree is only traversed once when detecting the collision between them and a candidate value waiting for insertion. This effectively reduces the repetitive collision detection. The corresponding node insertion operations of our CoTree are shown in Algorithm 2.

The positions to insert new nodes are the leaves of the tree, and we exploit an index i to point its last node (step 1 in Algorithm 2). For each candidate κ_j^u ($1 \leq u \leq \tau_\kappa$) of the j th subkey, we initialize an array $\mathbf{C}$ to record the number of collisions for all i nodes on the tree (step 3).

We further count the total number of collisions including two parts.

- 1) The collision between the v th node of tree $\mathbf{T}_{j-1}$ and the candidate κ_j^u . We can detect this by traveling and checking whether their collision value is within the given threshold τ_d (step 5). If the collision occurs, the function “*CheckCollision*” returns 1 for the collision flag $cFlag$. Otherwise, it returns 0.
- 2) The number of collisions the v th node inherits from its parent node. To exploit all collisions, our CoTree passes

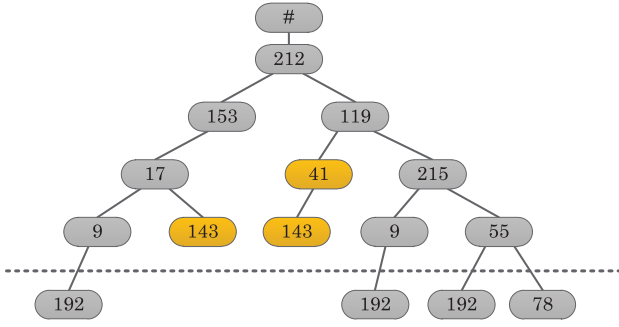


Fig. 3. CoTree containing candidates of the first five subkeys.

TABLE I
STATUS OF CoTREE CORRESPONDING TO FIG. 3

Identifier	Candidate	Parent	Layer	First Child	Degree
0	#	#	0	1	1
1	212	0	1	2	2
2	153	1	2	4	1
3	119	1	2	5	2
4	17	2	3	7	2
5	41	3	3	9	1
6	215	3	3	10	2
7	9	4	4	12	1
8	143	4	4	0	0
9	143	5	4	0	0
10	9	6	4	13	1
11	55	6	4	14	2
12	192	7	5	0	0
13	192	10	5	0	0
14	192	11	5	0	0
15	78	11	5	0	0

the number of collisions as a parameter in array $\mathbf{C}$ for each path from root to leaf. Obviously, the field “Parent” of the v th node makes it easy for us to locate its parent on the tree and inherit the corresponding number of collisions.

The total number of collisions we record into $\mathbf{C}_v$ corresponding to the v th node is the sum of these above two parts of collisions (step 6).

Finally, if the v th node is a leaf, we check the number of collisions $\mathbf{C}_v$ between κ_j^u and the nodes on this path from the second layer to it (step 7). If $\mathbf{C}_v = j - 1$, i.e., all collisions happen, we add $(\kappa_{j-1}^u, v, j, 0, 0)$ as its child after increasing i' by 1 (step 8). If the degree of the v th node is 1, i.e., this new leaf is its first child, we then set its field “First Child” as i' (steps 9–11). In this case, the corresponding information of the tree $\mathbf{T}_{j-1}$ for a node insertion is updated. We obtain the collision tree $\mathbf{T}_j$ after considering all candidates of the j th subkey within threshold τ_κ (step 15).

D. Node Removal

Suppose that a CoTree with only 212 as its candidate of the first subkey is as shown in Fig. 3 and the corresponding array is shown in Table I. Each row containing five fields in this table is a node as we defined in Section IV-A.

To facilitate the collision detection, node insertion is a *top-down* procedure (as we explained in Section IV-C). However,

Algorithm 3: Node Removal Operations

Input: Collision tree $\mathbf{T}_j$.
Output: The updated collision tree $\mathbf{T}_j$.

```

1  $i \leftarrow \text{LastNonLeafNode}(\mathbf{T}_j)$ ;
2 while  $i \geq 2$  do
3   if  $\text{NodeDegree}(\mathbf{T}_j, i) = 0$  then
4      $\text{temp}_1 \leftarrow \text{GetParent}(\mathbf{T}_j, i)$ ;
5      $\mathbf{T}_j \leftarrow \text{AdjustDegree}(\mathbf{T}_j, \text{temp}_1)$ ;
6     if  $\text{NodeDegree}(\mathbf{T}_j, \text{temp}_1) = 0$  then
7        $\mathbf{T}_j \leftarrow \text{SetChildEmpty}(\mathbf{T}_j, \text{temp}_1)$ ;
8     end
9      $\mathbf{T}_j \leftarrow \text{MoveChildren}(\mathbf{T}_j, \text{temp}_1 + 1)$ ;
10     $\mathbf{T}_j \leftarrow \text{RemoveNode}(\mathbf{T}_j, i)$ ;
11    if  $\text{NodeDegree}(\mathbf{T}_j, i) > 0$  then
12       $\text{temp}_2 \leftarrow \text{GetFirstChild}(\mathbf{T}_j, i)$ ;
13       $\mathbf{T}_j \leftarrow \text{MoveParents}(\mathbf{T}_j, \text{temp}_2)$ ;
14    end
15  end
16   $i = i - 1$ ;
17 end

```

if we traverse the tree and remove the nodes unsatisfying collision conditions in this model, all branches from root to leaves have to be traversed once. Obviously, this pulls our CoTree back to the worst case. In this case, we launch a *bottom-up* procedure to optimize this. The bottom-up node removal means that starting from the last node on the penultimate layer, we traverse the collision tree $\mathbf{T}_j$ from the back to the front, and the nodes without any branch are deleted (i.e., CoTree frees their space). The corresponding node removal operations are shown in Algorithm 3.

First, we record the last node on the $(j-1)$ th layer of CoTree $\mathbf{T}$ to i in step 1, and begin the bottom-top node removal. If this node has no branch (step 2), we find its parent with temp_1 as the identifier and adjust the corresponding degree by subtracting 1 (steps 4 and 5). If this i th node is the only child, we set its parent’s “First Child” field to -1 , which means that this node has no child and is waiting for removal later. Otherwise, its parent has several branches, its nearest brother becomes the new first child and we set the first child of its parent to i rather than $i - 1$ (steps 6–8). Since when we remove the i th node, its nearest brother (i.e., the $(i + 1)$ th node) becomes the new i th node in this case.

Consequently, the first children of all nonleaf nodes behind the parent of the i th node should be moved forward one position before removing it (steps 9 and 10). For example, all the nonleaf nodes 214, 9, 9, and 55 behind the parent 41 of the 9th node 143 should be moved forward one position when it is removed (as shown in Table I). If the new i th node has child (i.e., it is not a leaf), the field “Parent” for all nodes after its first child should be moved forward one position (steps 11–14). This can be achieved by subtracting them by 1. For example, the 10th node 9 becomes the new 9th node after removing the 9th node 143 in Table I, the parents from its child 192 to the last node 78 need to be moved forward one position.

The above operations guarantee the correctness of the relationship among the remaining nodes on our CoTree before and after removing nodes. The variable i always indicates the node

to be considered. It only needs to move forward one position no matter whether this node is deleted or not (step 15). All nodes unsatisfying the collision conditions are removed after the iteration. For example, all yellow nodes in Fig. 3 unsatisfying the collision conditions are removed, and the remaining gray nodes constitute the updated collision tree in this case.

E. Collision Chain Extraction

All collision chain detection algorithms will eventually extract the collision chains as full-key candidates for the latter verification. For example, the plaintext can be encrypted by using these key candidates according to certain rules (e.g., their ranked probabilities or number of collisions). If the corresponding ciphertext is generated in some encryption, we will consider the candidate as the correct key. The existing collision-optimized attacks extract collision chains once after traversing all candidates for each subkey within threshold τ_k , and the new chains are saved in the array. However, the extraction of collision chains in our CoTree is performed only once after finishing the node insertion of the final subkey (see step 9 in Algorithm 1), which will not bring too much computation.

F. Complexity

One key advantage of collision-optimized attacks is that their runtime is independent of their key rank in the key enumerations [21], but only depends on collision conditions. For example, a key ranking at 2^{20} or 2^{60} consumes the same time when transforming the given candidate space into a collision one under a certain τ_d if it falls within the given threshold τ_k . The remaining collision space after transformation is also the same. The only difference is that the latter may take more time in key verification since the key ranks more backward according some rules (e.g., their probability products).

However, the size of collision space in a collision-optimized attack under the same thresholds τ_d and τ_k varies greatly in different attacks, making the corresponding complexity change dynamically. Here, we first analyze the space complexity of CoTree in two extreme cases.

- 1) There is only one chain on CoTree. CoTree not only consumes more space but also consumes more time compared to FCC and LCD in this case. However, this makes no sense because the key is usually unrecoverable.
- 2) There are a total of

$$S(n, \tau_k) = \frac{(1 - \tau_k^n)}{1 - \tau_k} \quad (9)$$

nodes on our CoTree (i.e., it is a full τ_k tree). Here, n is 16 for AES-128 since it includes 16 subkeys. There will be at most τ_k^n chains for FCC and LCD, and the space consumed is $16 \cdot \tau_k^n$ in this case.

Suppose that each field is with the same size, the more required space of LCD and FCC compared to CoTree is

$$G(\tau_k) = 16 \cdot \tau_k^{15} - 5 \cdot \frac{(1 - \tau_k^{16})}{1 - \tau_k} \quad (10)$$

in the second extreme case for AES-128 with $L = 16$ subkeys, since there are five fields for each node on our CoTree.

Obviously

$$G(\tau_k) = \frac{16 \cdot \tau_k^{15}(\tau_k - 1) - 5 \cdot (\tau_k^{16} - 1)}{\tau_k - 1} \quad (11)$$

and this gives a new derivative function $G'(\tau_k)$ as

$$G'(\tau_k) = 11 \cdot \tau_k^{16} - 16 \cdot \tau_k^{15}. \quad (12)$$

Obviously, “ $\tau_k \geq 1.45$ ” well guarantees that our CoTree consumes less space than FCC and LCD if it is a full τ_k tree. Although CoTree is not always a full tree satisfying the above equations, this space advantage becomes more significant with the increase of τ_d and τ_k .

It is noteworthy that the advantage we want to emphasize here is not space complexity, but the time complexity. It is also more intuitive and easier to compare. First, when considering the collisions between the candidates of the $(i + 1)$ th subkey and the candidates of the previous i subkeys on the current tree, LCD only considers the case where several chains share the longest prefixes (i.e., their first $i - 1$ subkeys are with the same candidates). Our CoTree considers all possible prefixes thus it can be more efficiently alleviate the repetitive collision detection during each node insertion. Moreover, the bottom-up procedure guarantees that we should check all nodes only once in node removal operations at the worst case, and this only happens after all nodes are inserted. Therefore, most of the computation of our CoTree takes place in node insertion operations since it traverses all nodes on the tree to check their collisions with each candidate within τ_k of the subkey currently under consideration. Obviously, the larger the threshold, the less time consumption of CoTree compared to FCC and LCD, and this advantage will be further verified in our experiments in Section VI.

V. OPTIMIZATION

We provided basic operations of CoTree in Section IV. Actually, a lot of strategies can be introduced to optimize it. An example is, to maintain the ascending order of node identifiers on the tree, we detect the collisions between a candidate of the j th subkey and all nodes on the tree, then consider the next candidate in Algorithm 2, rather than the collisions between nodes on a branch and all the candidates of this coming subkey. This also facilitates the removal of nodes unsatisfying the collision conditions given in Section IV-D. The nodes of our CoTree will be traversed τ_k times in this case just like FCC and LCD. Another example is, the latter node removal will be performed only once as explained in Section IV-F. It is obvious that these operations will significantly reduce the repetitive collision detection compared to FCC and LCD and accelerate the candidate space transformation. We will further optimize our CoTree in several ways to enhance its performance in Sections V-A–V-B.

A. Branch Removal

The advantage of *bottom-up* procedure in Algorithm 3 is that to check whether all nodes on the tree satisfy the collision conditions, it only needs to traverse them once. This effectively alleviates the repetitive collision detection, especially for prefix

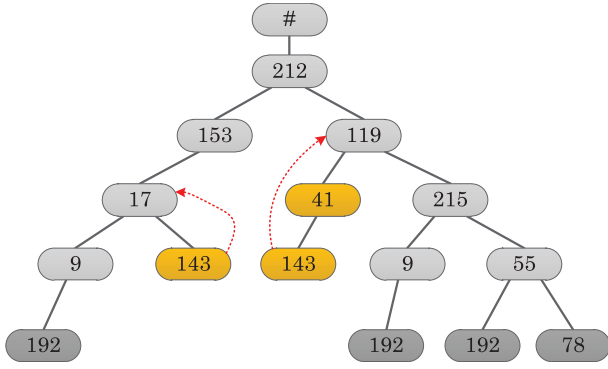


Fig. 4. Recursive removal of branches unsatisfying collision conditions.

subchains sharing the first several subkeys candidates. Since it is unnecessary to traverse the CoTree when removing nodes. Specifically, if a node on the penultimate layer has one or several children (i.e., leaves), this well illustrates that all nodes from its child (or children) to the root well satisfy the collision conditions. Therefore, we only need to judge the state of the penultimate nodes. If a node does not have any leaf node as its child, this branch should be removed recursively from the bottom to the root.

To illustrate the above node removal optimization, we give the corresponding diagram of the tree traversal in Fig. 4. Nodes 55 and 9 have children, which means that the nodes from them to the root and children all well satisfy the collision conditions, so we only need to detect their states rather than traverse their corresponding branches from bottom to top (i.e., to the root). Node 143 has no child, and we need to determine whether its parent node 41 has only one child before its removal. If so, this parent node should also be removed, too. The process proceeds recursively until node 119, since it has a child 215 and the recursion stops. It is obvious that this mechanism is more efficient than Algorithm 3 and it can avoid the traversal of some nodes, and this is especially effective for very large CoTree-s.

B. Fault Tolerance

It is usually a very high requirement that all collision values must be within the threshold τ_d . The performance of collision-optimized attacks can be significantly improved by performing fault tolerance on a small part of collisions. Here, an example of 120 pairs of collisions between 16 subkeys in the first round of AES-128 returned by CECA performed on randomly extracted 260 power traces leaked from AT89S52 microcontroller is shown in Fig. 5 in advance (see Section VI-B for more details). Specifically, we first give the collisions between the first subkey and the remaining 15 subkeys, then give the collisions between the second subkey and the remaining 14 subkeys, and repeat this until the collision between the 15th and 16th subkeys. In this case, the collision information of the j_1 th and j_2 th subkeys ($j_1 < j_2$) corresponds to the abscissa value “ $15 \cdot j_1 + (j_1 - 1) \cdot (j_1 - 2) + (j_2 - j_1)$,” while the ordinate represents the position of the collision value δ_{j_1, j_2} after ranking. For example, the abscissa of the collision value between the first two subkeys κ_1 and κ_2 is 1, and the ordinate is its ranking position 26. It can be seen that the deepest

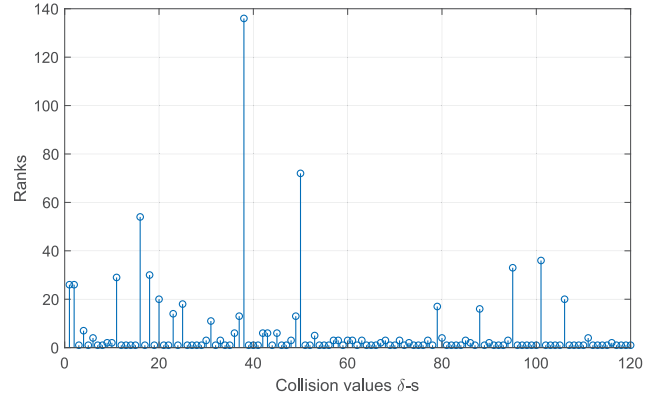


Fig. 5. Ranks of collision values among 16 subkeys of AES-128.

TABLE II
EXAMPLE OF COLLISION VALUES RANKED BEYOND THRESHOLD $\tau_d = 20$

Ranks	Collisions
≥ 20 and < 40	$\delta_{1,2}, \delta_{1,3}, \delta_{1,12}$ $\delta_{2,5}, \delta_{9,12}, \delta_{10,12}$
≥ 40 and < 60	$\delta_{2,3}$
≥ 60 and < 80	$\delta_{4,12}$
≥ 80 and < 140	$\delta_{3,12}$

collision value is 136, which means that, to recover the key, we need to set τ_d to at least 136 for all 120 pairs of collision values if without any fault tolerance. However, such a huge threshold is usually unacceptable when performing collision chain detection.

To solve the above problem, the collision-optimized attacks should contain fault tolerance mechanism to allow a small part of collision values beyond the threshold τ_d , thus reducing it. To illustrate this, we further give collision values corresponding to Fig. 5 but ranked beyond threshold 20 in Table II. It vividly shows that 6, 1, 1, and 1 collisions fall into the ranges: 20–40, 40–60, 60–80, and 80–140, respectively. Obviously, if we can identify the positions of $\delta_{2,3}$, $\delta_{3,12}$, and $\delta_{4,12}$ and perform fault tolerance on them, threshold τ_d can be reduced from 136 to 40, thus significantly reducing the collision space. This fully illustrates the significance of fault tolerance mechanisms on δ -s in collision-optimized attacks.

We achieve the above fault tolerance strategy by simply optimizing our CoTree as follows.

- 1) Based on Algorithm 2, an array C is additionally initialized to record the number of collisions for nodes when detecting their collision with each candidate within threshold τ_c of the subkey currently under consideration.
- 2) The collision information of each node is transferred to its children and accumulated. If the total number of collisions of a candidate is larger than the threshold

$$\tau_c = C_{\max} - \left\lceil \frac{j}{\epsilon} \right\rceil \quad (13)$$

we add it to the tree as a new leaf like [17] and [20]. Otherwise, we discard it.

- 3) We finally exploit the optimized Algorithm 3 introduced in Section V-A to remove the nodes unsatisfying the collision condition given in (13).

The symbol “ $C_{\max}$ ” in the above step 2) is the maximum number of collisions considering all chains, j indicates the j th subkey, and ϵ is fault tolerance coefficient adjusted according to the computing power of attacker. For example, if $C_{\max} = 18$ after collision detection of the first seven subkeys, $j = 7$ and $\epsilon = 3$, then $\tau_c = 18 - \lceil \frac{7}{3} \rceil = 15$ and we only maintain the chains with 15–18 pairs of collisions. Taking advantage of the fact that the chain of subkeys is almost one of the chains with the largest number of collisions, this strategy has achieved very high performance in fault tolerance.

VI. EXPERIMENTAL RESULTS

To illustrate the performance of CoTree, our experiments are performed on two serially implemented AES algorithms, which facilitate linear collision attacks.¹ Specifically, we first exploit CECA to optimize CPA and perform the first experiment on *DPA Contest V4.1* dataset implementing AES-256 in Section VI-A. Then, we further extend CECA to the profiled setting and exploit it to optimize TA, the corresponding experiment is performed on an AT89S52 microcontroller implementing AES-128 in Section VI-B. All the above experiments are performed on MATLAB R2016b on a DELL desktop computer with 6 Intel Core i5-9500 CPUs, 16-GB RAM, and a Windows 10 operating system. The widely exploited evaluation metric, success rate (SR) given in [25], is exploited to compare the performance of the existing collision-optimized attacks and our CoTree.

Unlike the well-applicable full-key rank estimation tools, most of the divide-and-conquer distinguishers lack theoretical tools to estimate the rank of a subkey (e.g., CPA), this shortcoming also occurs on the rank estimation of a collision value in collision attacks (e.g., CECA). Limited by this, it is difficult to tradeoff and determine thresholds in theory. However, large thresholds τ_d and τ_c will introduce more collision information to the target space τ_k of divide-and-conquer distinguisher to be optimized, thus pushing the remaining candidate space into a much smaller one, improving the SR and shortening the time consumption of key recovery. The fault tolerance coefficient ϵ in (13) is set to 3. We only perform 100 repetitions for each experiment since the evaluation is very time consuming. *cumulative distribution function (CDF)* on time consumption and key rank according to the number of collisions is exploited for better comparison.

A. Experiments on DPA Contest v4.1 Dataset

The widely used DPA Contest v4.1 open dataset [1] was targeted on the first round of AES-256 algorithm protected by RSM [16], and the algorithm was implemented on an Atmel ATmega-163 smart card connected to the SCA standard evaluation board named SASEBO-W. Due to the defect of the exploited random numbers, the implementation exists first-order leakage, i.e., can be directly attacked by first-order CPA and CECA (see [14] for detail). Here, we exploit CPA with Hamming weight model to exploit the leakage and further exploit traditional CECA

¹Linear collision attack is infeasible in parallel implementation as well explained in [8].

TABLE III
SRs UNDER DIFFERENT THRESHOLDS τ_d

τ_d ($\tau_k = 25$)	22	30	38	46	54	62
FCC, LCD, CoTree	0.43	0.49	0.44	0.41	0.41	0.47
τ_d ($\tau_k = 25$)	10	12	14	16	—	—
TOC	0.00	0.00	0.00	0.00	—	—
τ_d ($\tau_k = 25$)	6	8	10	12	—	—
FTC	0.00	0.00	0.00	0.00	—	—

to optimize it. We extract the best POI for each S-box and randomly extract 1100 power traces to perform the remaining experiments. We refer to the threshold setting method of Bogdanov et al. introduced in Section III-B and set τ_k for CPA to 25, thus we optimize a very huge candidate space with $25^{16} \approx 2^{74.3017}$ candidates.

For SR, we exploit the same fault tolerance strategy and consider the same fault tolerance coefficient $\epsilon = 3$ in (13) for FCC, LCD, and CoTree in our experiments. This guarantees that they achieve the same SR and obtain the same collision space. This means the same number of collision chains, thus facilitating the comparison of the strategies of *repetitive collision detection alleviation*. The SR is about 0 in TOC and FTC as given in Table III, but is even greater than 0.40 in FCC, LCD, and our CoTree. To improve the SR of TOC and FTC, we need a larger τ_d , which means higher complexity of collision detection. We draw a conclusion from this that flexible fault tolerance can significantly improve the attack SR in key recovery.

For *time consumption*, TOC, FTC, FCC, LCD, and our CoTree can transform very huge candidate space into a much smaller collision space quickly, and this even holds under very large τ_d for FCC, LCD, and our CoTree. The time consumption given in Fig. 6(d) and (e) vividly shows that our CoTree achieves significantly higher performance compared to FCC and LCD, and this trend becomes more significant under larger thresholds. About 50% experiments of FCC require more than 2000 s in Fig. 6(c), and about 40% experiments of the LCD require more than 300 s in Fig. 6(d), but there is no experiment consuming more than 200 s in our CoTree in Fig. 6(e). This trend becomes more significant under larger thresholds, since there will be significantly more collision chains satisfying the conditions and sharing the more nodes on the CoTree in this case, and the efficiency of alleviating repetitive collision detection will be further improved. However, compared to FCC, LCD, and CoTree, TOC and FTC can only work under very small thresholds τ_d and τ_k , since they only exploit 15 pairs of collisions and too many collision chains will satisfy this condition [see Fig. 6(a) and (b)].

For *collision space*, the collision chains in the collision-optimized attacks TOC, FTC, FCC, LCD, and CoTree after space transformation are ranked and successively verified according to their number of collisions as exploited in the previous work [18]. The key rank of the original CPA to be optimized is computed by the classical key rank estimation tool given in [21]. FCC, LCD, and CoTree exploit most of the collisions, and a small part of the remaining collisions are fault

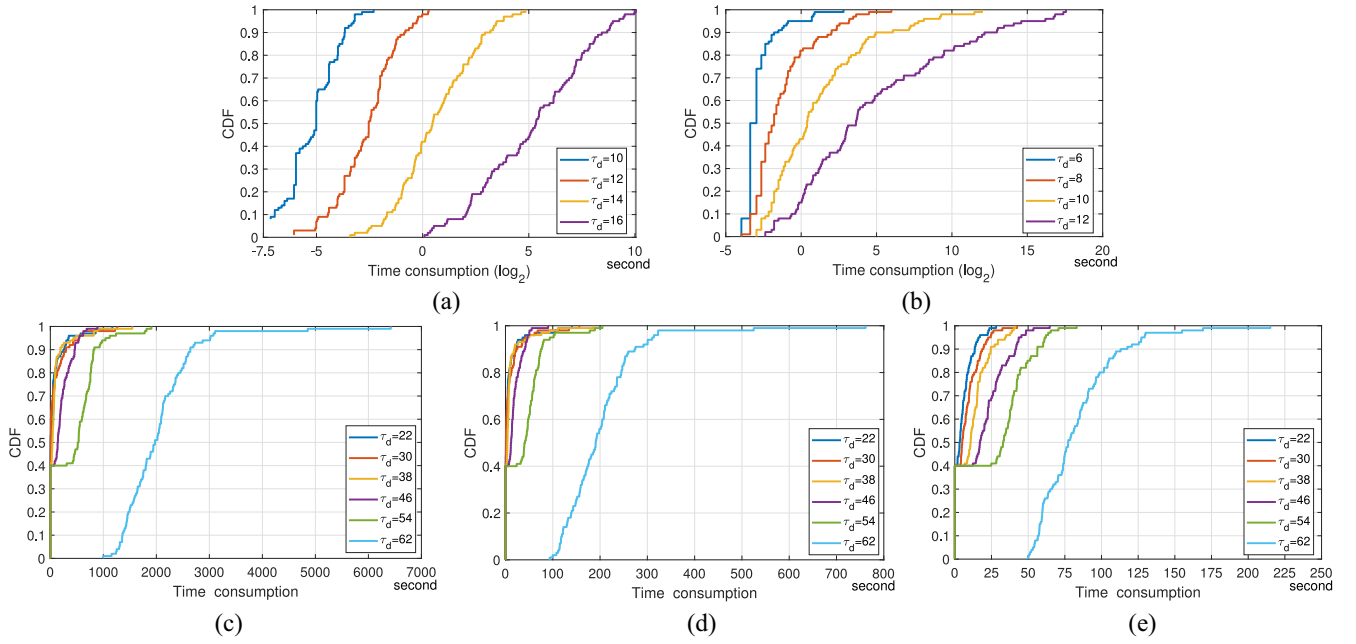


Fig. 6. Time consumption of TOC, FTC, FCC, LCD, and our CoTree under different thresholds τ_d .

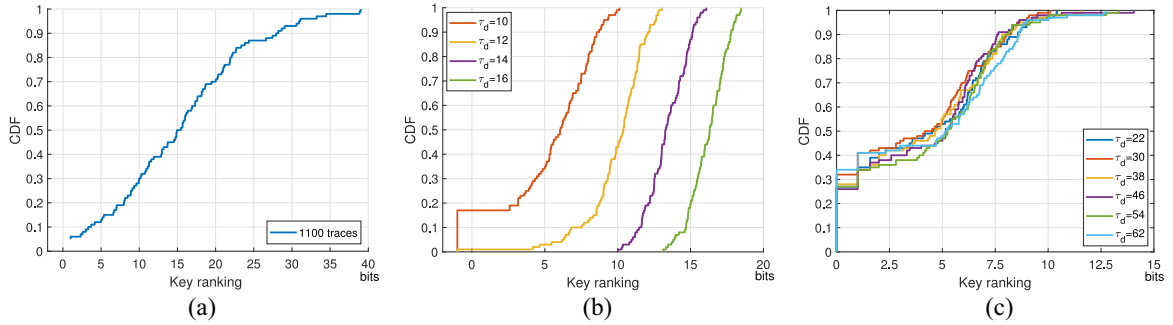


Fig. 7. Key rank in CPA, TOC, FCC, LCD, and CoTree under different thresholds τ_d .

tolerated. More collisions exploited mean smaller collision space, and they achieve a much smaller collision space. The key rank in chain space is also much better than the key rank estimated in the original candidate space and the verification will consume very limited time (as shown in Fig. 7). However, TOC and FTC only exploit 15 pairs of collisions, too many collision chains remained and a small increase on τ_d brings huge burden for collision detection. Moreover, there will be no chain remaining in FTC in most cases, so we do not give the experimental results here (the same conclusion can also be drawn in Section VI-B). Obviously, TOC and FTC are not suitable for the case that both thresholds τ_k and τ_d are very large.

Above all, FCC, LCD, and CoTree spend significantly less time on space transformation and leave a significantly smaller collision space under much larger thresholds compared with TOC and FTC. Our CoTree exploits a data structure very different from FCC and LCD, and it is a tree rather than simply an array exploited in both of them. It significantly reduces repetitive collision detection in FCC and LCD, and this trend becomes more significant under larger thresholds. We can draw

a conclusion that our CoTree achieves significantly higher SR and smaller collision space than TOC and FTC, and consumes significantly less time than FCC and LCD, thus it better alleviates the difficulty of key recovery of the existing collision-optimized attacks.

B. Experiments on an AT89S52 Microcontroller

Our second experiment is performed on an AES-128 algorithm implemented on an AT89S52 microcontroller. The operating frequency of this chip is 12 MHz. The sampling rate of our Picoscope 3000 is set to 125 MS/s. We acquire 50000 power traces and perform CPA to extract the best POI for each S-box in the first round. We exploit 50 traces to profile a template for each Hamming weight in TA and template for each intermediate value. This extends CECA to the profiled setting, and we exploit the profiled distinguisher

$$\arg \max_{\kappa_1, \kappa_2} \sqrt{\sum_{i=1}^n \sum_{j=1}^2 \left(t_j^i - \mathbf{m}_j^{\text{Sbox}(x_j^i \oplus \kappa_j)} \right)^2} \quad (14)$$

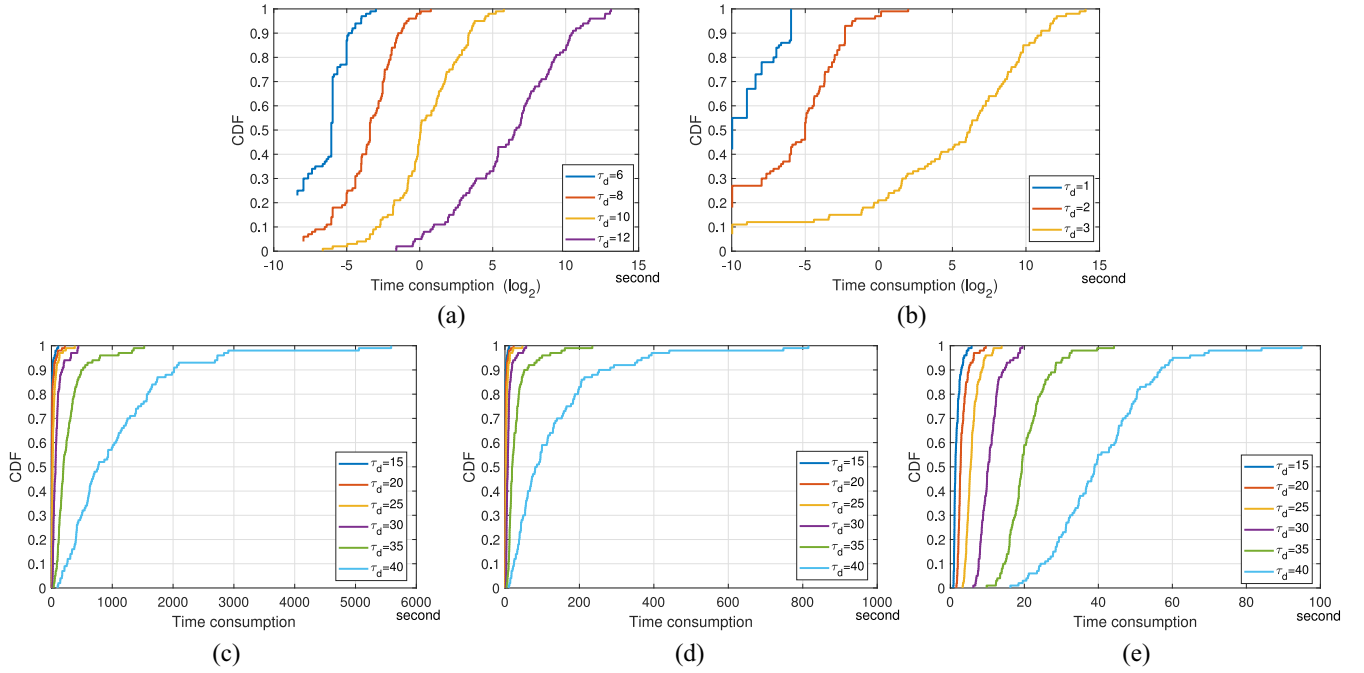


Fig. 8. Time consumption of TOC, FTC, FCC, LCD, and our CoTree under different thresholds τ_d .

as given in [17] and [18]. Here, x_j^i denotes the j th byte of the i th encrypted plaintext, t_j^i denotes the corresponding POI, and $\mathbf{m}_j^{\text{Sbox}(x_j^i \oplus \kappa_j)}$ denotes the profiled mean power consumption template corresponding to intermediate value with $x_j^i \oplus \kappa_j$ as the S-box input as defined in Section II. This profiled CECA will achieve performance much closer to TA compared to the original CECA, and their combination will be more meaningful in this case. We then randomly extract 260 power traces, and perform the remaining experiments in each repetition.

The threshold setting in collision-optimized attacks is not invariable. There are many factors affecting this (e.g., different target devices and power analysis methods), and transforming the candidates within the same given thresholds into the collision space will consume very different times in different scenarios. If we set the same thresholds for FCC, LCD, and CoTree as considered in Section VI-A, they will spend more time and even not work. Therefore, we set $\tau_\kappa = 25$ and optimize a huge candidate space with $25^{16} \approx 2^{74.3017}$ candidates just as Section VI-A, but consider a much smaller τ_d from 15 to 40. FTC's time consumption is even unbearable under $\tau_\kappa = 25$ and $\tau_d = 2$. Therefore, we adjust these two thresholds to $\tau_\kappa = 7$ and $\tau_d = 1, 2, 3$.

For SR, we exploit the same fault tolerance strategy and consider the same fault tolerance coefficient $\epsilon = 3$ in (13) for FCC, LCD, and CoTree in our experiments. Therefore, FCC, LCD, and our CoTree achieve the same SR (as shown in Table IV). Different from the results given in Section VI-A, they achieve higher SRs with the growth of τ_d . However, TOC only achieves a very low SR under $\tau_d = 30$ and no success is achieved in FTC. We draw the same conclusion as given in Section VI-A from this that flexible fault tolerance can significantly improve the attack SR in collision-optimized attacks.

TABLE IV
SRs UNDER DIFFERENT THRESHOLDS τ_d

τ_d ($\tau_\kappa = 25$)	15	20	25	30	35	40
FCC, LCD, CoTree	0.26	0.35	0.39	0.44	0.51	0.55
τ_d ($\tau_\kappa = 25$)	6	8	10	12	—	—
TOC	0.07	0.12	0.19	0.25	—	—
τ_d ($\tau_\kappa = 7$)	1	2	3	—	—	—
FTC	0.00	0.00	0.00	—	—	—

For *time consumption*, TOC and FTC are more time consuming than FCC, LCD, and our CoTree as shown in Fig. 8, since they exploit only 15 pairs of collisions and too many chains satisfying the collision conditions as we have explained before. Similar to the results given in Section VI-A, FTC is still infeasible under large thresholds τ_κ and τ_d . About 10% experiments of FCC require more than 2000 s in Fig. 8(c), and about 10% experiments of the LCD require more than 300 s in Fig. 8(d), but there is no experiment consuming more than 100 s in our CoTree in Fig. 8(e). Obviously, our CoTree consumes significantly less time than FCC and LCD, which vividly illustrates that it significantly alleviates the repetitive collision detection due to its efficient collision chain storage structure and detection algorithm. The efficiency of alleviating repetitive collision detection can be significantly improved when considering larger thresholds and more collision chains, since this will further promote the nodes sharing.

For *collision space*, FTC generates a very large number of chains satisfying the collision condition, but this situation does not last until the last subkey since almost no full-key candidates remain in most cases (e.g., 90% of repetitions). Therefore, we do not give its results in figures. Similar to Section VI-A, here the key rank of the original TA

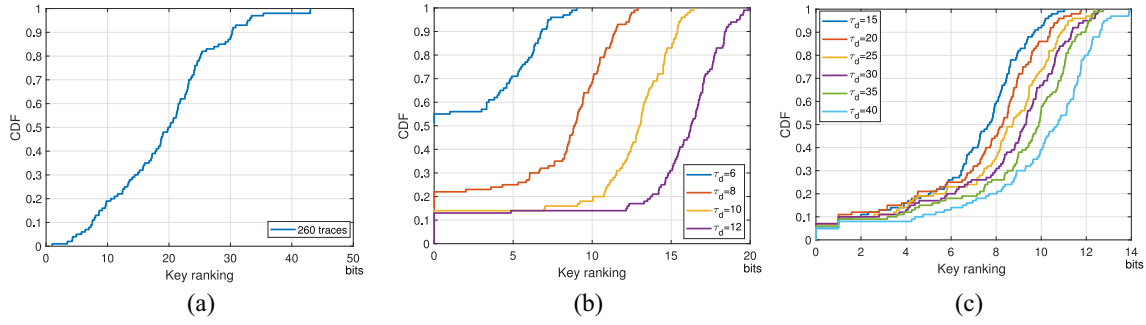


Fig. 9. Key rank in TA, TOC, FCC, LCD, and our CoTree under different thresholds τ_d .

to be optimized is also computed by the classical key rank estimation tool given in [21]. The key ranks in TOC, FCC, LCD, and our CoTree are given in Fig. 9(b) and (c) are much better than the original key ranks in TA given in Fig. 9(a). Key ranks in FCC, LCD, and CoTree under $\tau_d = 40$ are even better than these in TOC under $\tau_d = 12$. This well illustrates that we can achieve a smaller collision space if considering more collisions.

Above all, we can draw a conclusion that our CoTree achieves a much higher SR than TOC and FTC, and its time consumption is much less than the existing schemes. This fully illustrates that our CoTree significantly outperforms these works.

VII. CONCLUSION

To alleviate the repetitive collision detection of the existing collision-optimized attacks, we proposed a highly efficient collision detection tool named CoTree in this article. This new collision-optimized attack exploits tree data structure to store collisions and consumes significantly fewer storage space. Benefiting from the properties of tree, our CoTree exploits a top-down tree building procedure and a bottom-up branch removal procedure to significantly alleviate the serious repetitive collision detection. Our experiments verified that CoTree significantly outperforms the existing works, and this advantage becomes more significant when considering a much larger number of collisions.

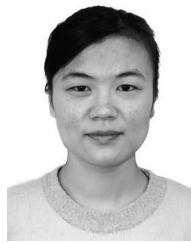
Collision-optimized attacks can attack the serially implemented cryptographic algorithms with S-box operations, such as AES, DES, PRESENT, etc., as explained in Section I-A, and push the limits of their conquerable candidate space. However, we still have some works to do in collision-optimized attacks except for introducing more efficient collision attacks (e.g., [9] and [28]) to further improve CECA. First, we will try to estimate the SR of divider-and-conquer attack and collision attack in theory to facilitate their thresholds determination, trade-offs, and complexity estimation, and compare the collision-optimized attacks in a more efficient and fair manner. Second, we will also consider different cryptographic implementations and more complex scenarios (e.g., large noise). Third, although our CoTree achieves performance much better than the existing works, some collision information is still repetitively detected, and we will further alleviate this in our future works. Finally, some operations like node insertion and removal, can be

processed simultaneously, and the parallel key search in our CoTree is also a very interesting topic. We will further solve the above problems in our future work.

REFERENCES

- [1] "DPA contest v4.1." Accessed: Jul. 7, 2023. [Online]. Available: <http://www.dpacontest.org/home/>
- [2] A. Bogdanov, "Multiple-differential side-channel collision attacks on AES," in *Proc. 10th Int. Workshop Cryptograph. Hardw. Embedded Syst.*, 2008, pp. 30–44.
- [3] A. Bogdanov and I. Kizhvatov, "Beyond the limits of DPA: Combined side-channel collision attacks," *IEEE Trans. Comput.*, vol. 61, no. 8, pp. 1153–1164, Aug. 2012.
- [4] E. Brier, C. Clavier, and F. Olivier, "Correlation power analysis with a leakage model," in *Proc. 6th Int. Workshop Cryptograph. Hardw. Embedded Syst.*, Cambridge, MA, USA, 2004, pp. 16–29.
- [5] S. Chari, J. R. Rao, and P. Rohatgi, "Template attacks," in *Proc. 4th Int. Workshop Cryptograph. Hardw. Embedded Syst.*, Redwood Shores, CA, USA, Aug. 2002, pp. 13–28.
- [6] Y. Ding et al., "A multiple sieve approach based on artificial intelligent techniques and correlation power analysis," *ACM Trans. Multimed. Commun. Appl.*, vol. 17, no. 2, p. 71, 2021.
- [7] T. Espitau, P. Fouque, B. Gérard, and M. Tibouchi, "Side-channel attacks on BLISS lattice-based signatures: Exploiting branch tracing against strongSwan and electromagnetic emanations in microcontrollers," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2017, pp. 1857–1874.
- [8] B. Gérard and F. Standaert, "Unified and optimized linear collision attacks and their application in a non-profiled setting," in *Proc. 14th Int. Workshop Cryptograph. Hardw. Embedded Syst.*, 2012, pp. 175–192.
- [9] C. Glowacz and V. Grosso, "Optimal collision side-channel attacks," in *Proc. 18th Int. Conf. Smart Card Res. Adv. Appl.*, 2019, pp. 126–140.
- [10] V. Grosso, "Scalable key rank estimation (and key enumeration) algorithm for large keys," in *Proc. 17th Int. Conf. Smart Card Res. Adv. Appl.*, 2018, pp. 80–94.
- [11] P. C. Kocher, "Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems," in *Proc. 16th Annu. Int. Cryptol. Conf. Adv. Cryptol.* 1996, pp. 104–113.
- [12] Y. Li, S. Wang, Z. Wang, and J. Wang, "A strict key enumeration algorithm for dependent score lists of side-channel attacks," in *Proc. 16th Int. Conf. Smart Card Res. Adv. Appl.*, 2017, pp. 51–69.
- [13] A. Moghimi, G. Irazoqui, and T. Eisenbarth, "CacheZoom: How SGX amplifies the power of cache attacks," in *Proc. 19th Int. Conf. Cryptograph. Hardw. Embedded Syst.*, 2017, pp. 69–90.
- [14] A. Moradi, S. Guilley, and A. Heuser, "Detecting hidden leakages," in *Proc. 12th Int. Conf. Appl. Cryptograph. Netw. Security*, 2014, pp. 324–342.
- [15] A. Moradi, O. Mischke, and T. Eisenbarth, "Correlation-enhanced power analysis collision attack," in *Proc. 12th Int. Workshop Cryptograph. Hardw. Embedded Syst.*, 2010, pp. 125–139.
- [16] M. Nassar, Y. Souissi, S. Guilley, and J. Danger, "RSM: A small and fast countermeasure for AES, secure against 1st and 2nd-order zero-offset SCAs," in *Proc. Design, Autom. Test Europe Conf. Exhibition*, 2012, pp. 1173–1178.
- [17] C. Ou, S.-K. Lam, and G. Jiang, "The science of guessing in collision-optimized divide-and-conquer attacks," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 40, no. 6, pp. 1039–1051, Jun. 2021.

- [18] C. Ou, S. Lam, C. Zhou, G. Jiang, and F. Zhang, "A lightweight detection algorithm for collision-optimized divide-and-conquer attacks," *IEEE Trans. Comput.*, vol. 69, no. 11, pp. 1694–1706, Nov. 2020.
- [19] C. Ou, Z. Wang, D. Sun, and X. Zhou, "Group collision attack," *IEEE Trans. Inf. Forensics Security*, vol. 14, pp. 939–953, 2019.
- [20] C. Ou, C. Zhou, S.-K. Lam, and G. Jiang, "Multiple-differential mechanism for collision-optimized divide-and-conquer attacks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 418–430, 2021.
- [21] R. Poussier, F. Standaert, and V. Grosso, "Simple key enumeration (and rank estimation) using histograms: An integrated approach," in *Proc. 18th Int. Conf. Cryptograph. Hardw. Embedded Syst.*, 2016, pp. 61–81.
- [22] C. Rechberger and E. Oswald, "Practical template attacks," in *Proc. 5th Int. Workshop Inf. Security Appl.*, 2004, pp. 440–456.
- [23] K. Schramm, G. Leander, P. Felke, and C. Paar, "A collision-attack on AES: Combining side channel- and differential-attack," in *Proc. 6th Int. Workshop Cryptograph. Hardw. Embedded Syst.*, 2004, pp. 163–175.
- [24] K. Schramm, T. J. Wollinger, and C. Paar, "A new class of collision attacks and its application to DES," in *Proc. 10th Int. Workshop Fast Softw. Encrypt.*, 2003, pp. 206–222.
- [25] F. Standaert, T. Malkin, and M. Yung, "A unified framework for the analysis of side-channel key recovery attacks," in *Proc. 28th Annu. Int. Conf. Theory Appl. Cryptograph. Techn. Adv. Cryptol.* 2009, pp. 443–461.
- [26] D. Wang, A. Wang, and X. Zheng, "Fault-tolerant linear collision attack: A combination with correlation power analysis," in *Proc. 10th Int. Conf. Inf. Security Pract. Exp.*, 2014, pp. 232–246.
- [27] W. Wang, Y. Yu, F. Standaert, J. Liu, Z. Guo, and D. Gu, "Ridge-based DPA: Improvement of differential power analysis for nanoscale chips," *IEEE Trans. Inf. Forensics Security*, vol. 13, pp. 1301–1316, 2018.
- [28] A. Wiemers and D. Klein, "Entropy reduction for the correlation-enhanced power analysis collision attack," in *Proc. 13th Int. Workshop Security Adv. Inf. Comput. Security*, 2018, pp. 51–67.



Kexin Qiao received the bachelor's degree in mathematics from Shandong University, Jinan, China, in 2012, and the Ph.D. degree in computer science from the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China, in 2017.

She is currently an Assistant Professor with the School of Cyber Science and Engineering, Beijing Institute of Technology, Beijing. Her research interests include side-channel attacks, cryptanalysis, and cryptographic security implementation.



Shihui Zheng received the Ph.D. degree from Shandong University, Jinan, China, in 2006.

From 2006 to 2008, she was a Postdoctoral Fellow with the School of Information Engineering, Beijing University of Posts and Telecommunications (BUPT), Beijing, China. In 2008, she joined the School of Cyberspace Security and the National Engineering Laboratory for Disaster Backup and Recovery, BUPT. Her current research interests are cryptographic scheme design and side-channel attacks.



Changhai Ou received the B.S. degree in computer science and technology from the School of Computer and Information Technology, Beijing Jiaotong University, Beijing, China, in 2013, and the Ph.D. degree from the School of Cyber Security, University of Chinese Academy of Sciences (i.e., Institute of Information Engineering, Chinese Academy of Sciences), Beijing, in July 2018.

He was then a Research Fellow with the School of Computer Science and Engineering, Nanyang Technological University, Singapore. He is currently a Full Professor with the School of Cyber Science and Engineering, Wuhan University, Wuhan, Hubei, China. His current research interests include hardware and embedded system security, side-channel attacks, applied cryptography, AI security, and signal processing.



Siew-Kei Lam (Senior Member, IEEE) received the B.A.Sc., M.Eng., and Ph.D. degrees from the School of Computer Engineering, Nanyang Technological University, Singapore.

His Ph.D. dissertation investigated techniques to automatically generate custom instructions for reconfigurable processors. He was a Visiting Research Fellow with Imperial College London, London, U.K.; the University of Warwick, Coventry, U.K.; and RWTH Aachen University, Aachen, Germany. He is currently an Assistant Professor with

the School of Computer Science and Engineering, Nanyang Technological University. His research investigates methods for realizing custom computing solutions in embedded systems.

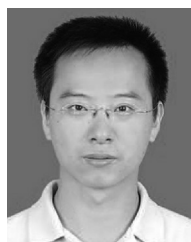


Debiao He received the B.S. degree from the Department of Mathematics, Shandong Normal University, Jinan, China, in 2003, and the Ph.D. degree in applied mathematics from the School of Mathematics and Statistics, Wuhan University, Wuhan, China, in 2009.

He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University. He has authored or coauthored more than 100 research papers in refereed international journals and conferences, such as the IEEE TRANSACTIONS

ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, and Usenix Security Symposium. His main research interests include cryptography and information security, in particular cryptographic protocols.

Prof. He was the recipient of the 2018 IEEE SYSTEMS JOURNAL Best Paper Award and the 2019 IET Information Security Best Paper Award. His work has been cited more than 10 000 times at Google Scholar. He is on the editorial board of several international journals, such as the *ACM Distributed Ledger Technologies: Research and Practice*, *Frontiers of Computer Science*, and IEEE TRANSACTIONS ON COMPUTERS.



Fan Zhang (Member, IEEE) received the Ph.D. degree from the Department of Computer Science and Engineering, University of Connecticut, Storrs, CT, USA, in 2012.

He is currently a Full Professor with the School of Cyber Science and Technology, College of Computer Science and Technology, Zhejiang University, Hangzhou, China, where he is also affiliated with the College of Information Science and Electronic Engineering. His research interests include hardware security, system security, cryptography, and computer architecture.